

iOS APPLICATION ANALYSIS LAB

User Manual

Table of contents

1. Objectives.....	3
2. Prerequisites.....	3
3. Lab Environment Setup.....	3
3.1 Host System Requirements.....	4
3.2 Verify Host System.....	4
3.3 Install Oracle VirtualBox.....	6
3.4 Obtain the Provided Virtual Machine.....	11
3.5 Import the OVA.....	12
3.6 Start the Virtual Machine.....	14
3.7 Log in to Ubuntu.....	15
3.8 Verify the Lab Environment.....	16
4. Introduction to iOS Application Static Analysis.....	17
4.1 What is an IPA File?.....	17
4.2 IPA File Structure.....	17
4.3 iOS Application Bundle Structure.....	17
4.4 Important Files and Directories.....	18
5. Static Analysis of an iOS Application.....	19
5.1 Identify the IPA File.....	19
5.2 Inspect the IPA Archive.....	20
5.3 Extract the IPA.....	20
5.4 Locate the Payload Directory.....	20
5.5 Locate the Application Bundle.....	21
5.6 Analyze the Application Bundle.....	22
5.7 Analyze Info.plist.....	22
5.7.1 Display Info.plist.....	23
5.7.2 Analyze Application Metadata.....	23
5.7.3 Analyze App Transport Security.....	23
5.7.4 Analyze URL Schemes.....	24
5.7.5 Analyze Permission-related Entries.....	24
5.7.6 Analyze iOS Version Information.....	24
5.8 Search for Sensitive Files.....	25
5.9 Identify the Main Mach-O Binary.....	25
5.10 Analyze Mach-O Headers.....	25
5.11 Analyze Imported Symbols.....	26
5.12 Extract Strings.....	26
5.13 Search for Potential Secrets.....	27
5.14 Extract URLs.....	27

5.15 Analyze Embedded Frameworks.....	28
5.16 Analyze Cryptographic APIs.....	28
5.17 Analyze Storage APIs.....	28
5.18 Analyze WebView APIs.....	29
5.19 Analyze Network and TLS APIs.....	29
5.20 Analyze PIE Protection.....	29
5.21 Analyze Encryption Information.....	30
5.22 Analyze Stack Protection.....	30
6. Identification of Potential Security Weaknesses.....	30
6.1 Sensitive Information Exposure.....	31
6.2 Hardcoded Secrets.....	32
6.3 Insecure Network Configuration.....	33
6.4 Weak Cryptographic Usage.....	33
6.5 Insecure Storage Indicators.....	34
6.6 WebView Security Indicators.....	35
6.7 Sensitive Files.....	36

1. Objectives

The objective of this laboratory is to introduce the fundamentals of **iOS application static analysis**.

After completing this laboratory, the user should be able to:

- Identify and extract an IPA file.
- Locate the application bundle and **Info.plist**.
- Identify the main Mach-O executable.
- Analyze strings, URLs, imported APIs, and embedded frameworks.
- Review cryptographic, storage, WebView, and network-related APIs.
- Examine basic binary protection mechanisms.
- Identify potential security weaknesses from static-analysis observations.

2. Prerequisites

The user should have basic familiarity with:

- Linux command-line usage.

Note: No previous iOS security-analysis experience is required. The analysis environment is already prepared inside the provided virtual machine. The user does not need to install the iOS analysis tools individually unless specifically instructed.

3. Lab Environment Setup

The laboratory uses a pre-configured Ubuntu Linux virtual machine packaged as a VirtualBox-compatible OVA. Architecture-specific versions are provided for AMD64 and ARM64 host systems.

The user must have **Oracle VirtualBox** installed on a compatible Windows or Linux host system. The provided OVA contains the pre-configured laboratory environment required for the exercise.

3.1 Host System Requirements

The host system should meet the following minimum requirements.

Requirement	Minimum Requirement
Operating System	Windows 10/11 64-bit or Linux 64-bit
CPU Architecture	AMD64/x86_64 or ARM64, as supported by the provided VM
RAM	8 GB or more
Free Storage	50 GB or more
Hardware Virtualization	Enabled
Internet	Required
Virtualization Software	Oracle VirtualBox

Important: Two architecture-specific virtual machines are provided. Select the OVA that matches the CPU architecture of the host system.

3.2 Verify Host System

Before downloading and importing the virtual machine, verify that your computer meets the requirements.

Windows Host

Step 1: Check System Architecture

- Open: **Settings** → **System** → **About**
- Locate **System type**.
- Identify the processor architecture.

Select	System type
AMD64 OVA	x64-based processor
ARM64 OVA	ARM-based processor

System type 64-bit operating system, x64-based processor

Step 2: Check Available Memory

- Open: **Task Manager** → **Performance** → **Memory**
- Verify that the system has at least **8 GB of RAM**.
- **Expected Result:**
The system should have 8 GB or more of installed memory.

Step 3: Check Available Storage

- Open: **File Explorer** → **This PC**
- Check the available space on the drive where the OVA and virtual machine will be stored.
- Ensure that at least **50 GB of free storage** is available.
- **Expected Result:**
Sufficient free storage should be available for downloading, importing, and running the virtual machine.

Step 4: Check Hardware Virtualization

- Open: **Task Manager** → **Performance** → **CPU**
- Locate **Virtualization**.
- **Expected Result:**
Virtualization should be shown as **Enabled**.

Linux Host

- Open a terminal.
- **Check Architecture**

○ `uname -m`

● **Expected Result:**

Output	Select
x86_64	AMD64 OVA
aarch64	ARM64 OVA

```
cdac@cdac-ThinkStation-P330:~$ uname -m
x86_64
cdac@cdac-ThinkStation-P330:~$
```

Figure 3-2: Linux host architecture verification

3.3 Install Oracle VirtualBox

Oracle VirtualBox is used to run the provided architecture-specific laboratory virtual machine.

[Oracle VirtualBox Downloads](#)

Windows Host

Step 1: Download VirtualBox

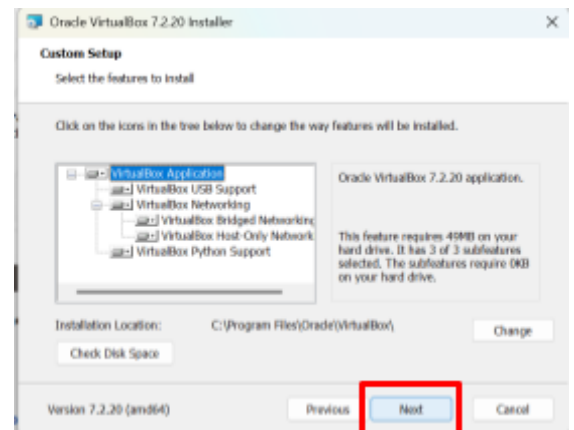
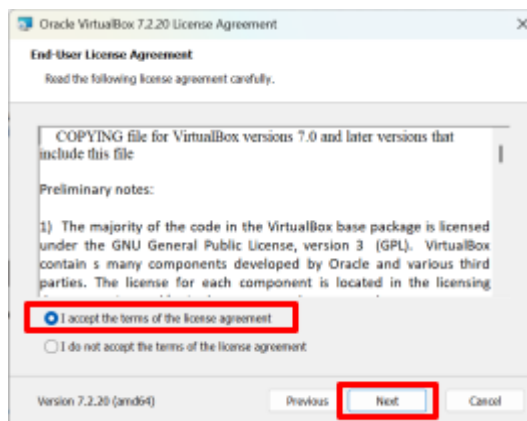
- Open the VirtualBox download page.
- Select **Windows Hosts**.
- Download the installer.



Figure 3-3: VirtualBox download page

Step 2: Install VirtualBox

- Open the downloaded installer.
- Follow the installation wizard using the default options.
- Select **Install** when prompted.
- Allow required Windows permission prompts.



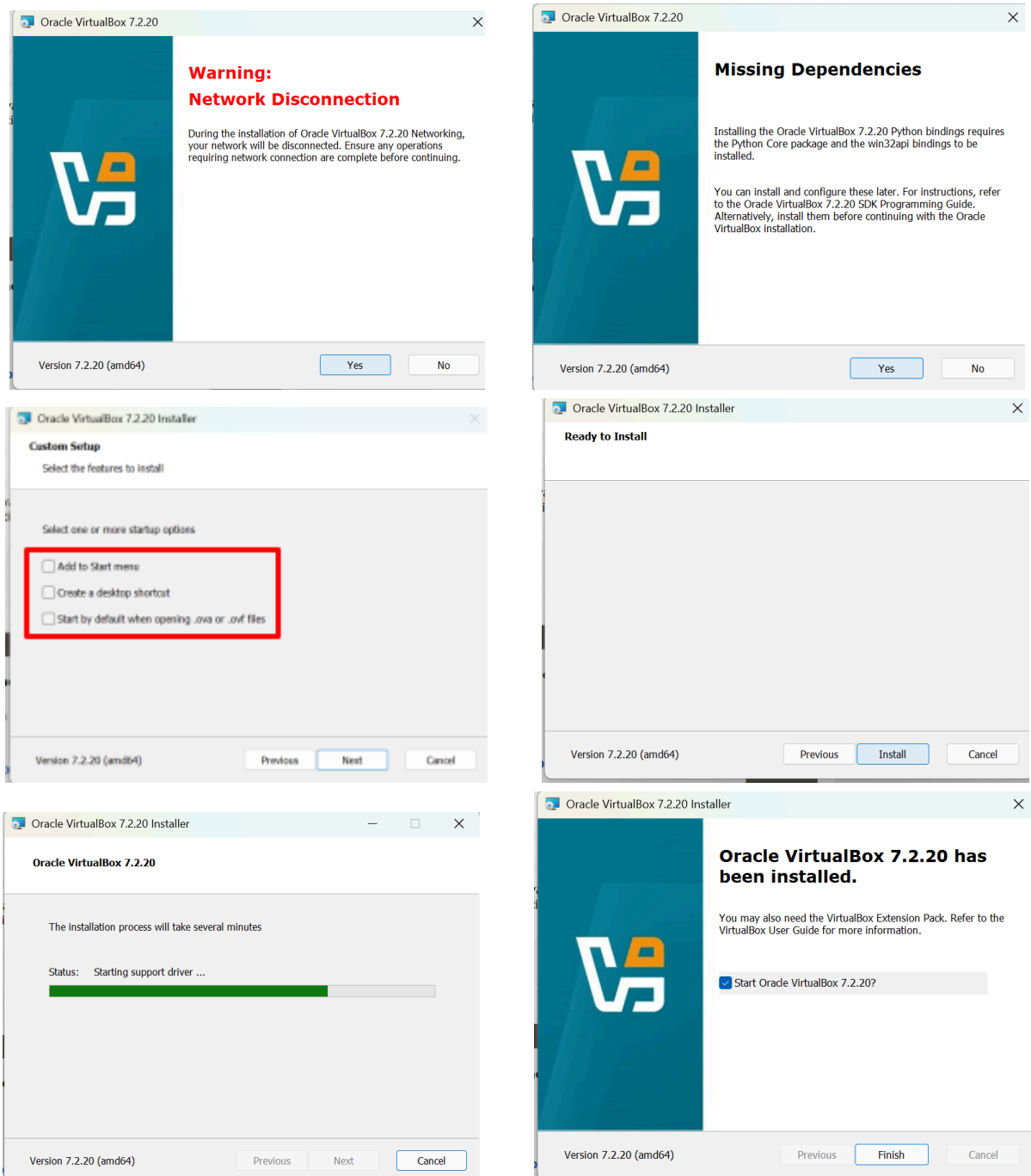


Figure 3-4: VirtualBox installation wizard

Step 3: Launch VirtualBox

- Open **Oracle VirtualBox** from the Start menu or desktop.
- Confirm that **VirtualBox Manager** opens.

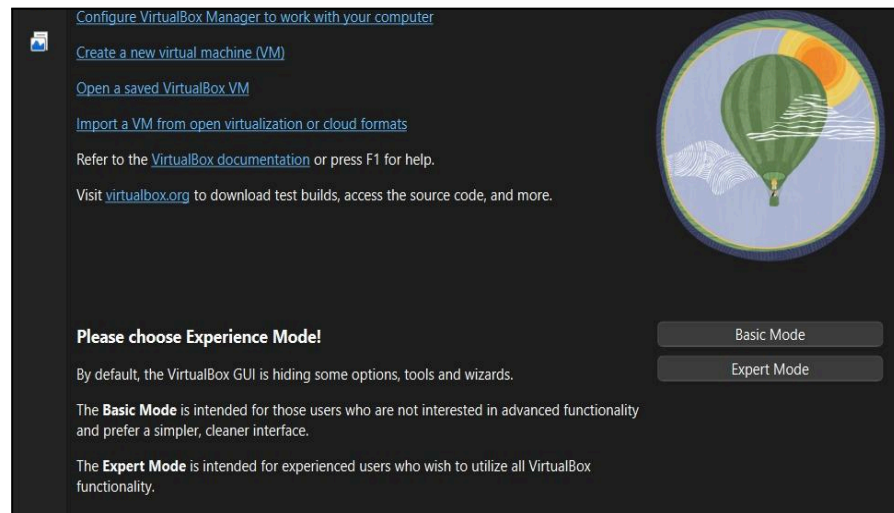


Figure 3-5: Oracle VirtualBox Manager

Linux Host

Step 1: Download VirtualBox

- Open the VirtualBox download page.
- Select **Linux distributions**.
- **Select the package matching the physical host Linux distribution.**
- Download the package.

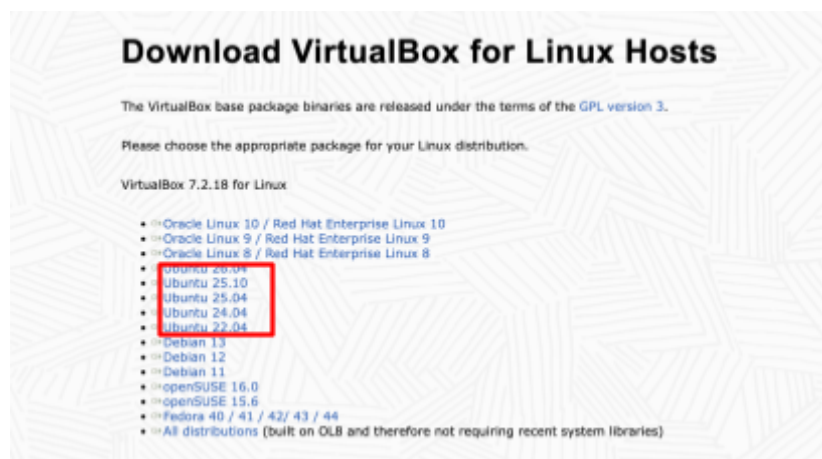


Figure 3-6: VirtualBox download page for Linux hosts

Step 2: Install VirtualBox

- Open **Files** → **Downloads**.
- Double-click the downloaded **.deb** package.

- Select **Install**.
- Enter the administrator password if prompted.



Figure 3-7: Downloaded VirtualBox package on Linux

Step 3: Launch VirtualBox

- Open the application menu.
- Select **Oracle VirtualBox**.
- Confirm that **VirtualBox Manager** opens.

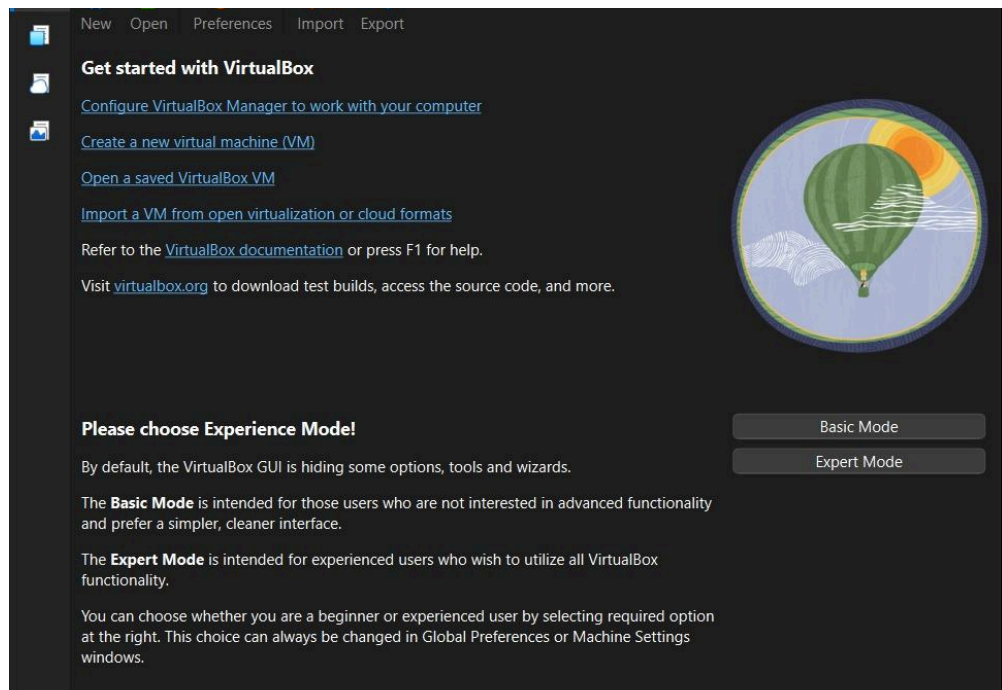


Figure 3-8: Oracle VirtualBox Manager before importing the lab VM

3.4 Obtain the Provided Virtual Machine

The laboratory environment is provided as a pre-configured OVA file.

Host Architecture	OVA File
Intel/AMD x86_64	iOS-Static-Lab-AMD64.ova AMD64 OVA File
ARM64	iOS-Static-Lab-ARM64.ova ARM64 OVA File

Step 1: Download the OVA

- Open the provided laboratory download location.
- Select the OVA corresponding to the host architecture.
- Download the file.
- Save it to a location with sufficient free storage.

Step 2: Verify the OVA

- Open the download location.
- Confirm that the selected OVA file is present.

Important: Do not extract, rename, or modify the OVA file.



Figure 3-9: Provided iOS Static Lab OVA file



Figure 3-10: Downloaded OVA file on Linux

Important: Do not extract or modify the OVA file.

3.5 Import the OVA

- Open **Oracle VirtualBox**.

Step 1: Open Import Appliance

- Select **File** → **Import Appliance**.

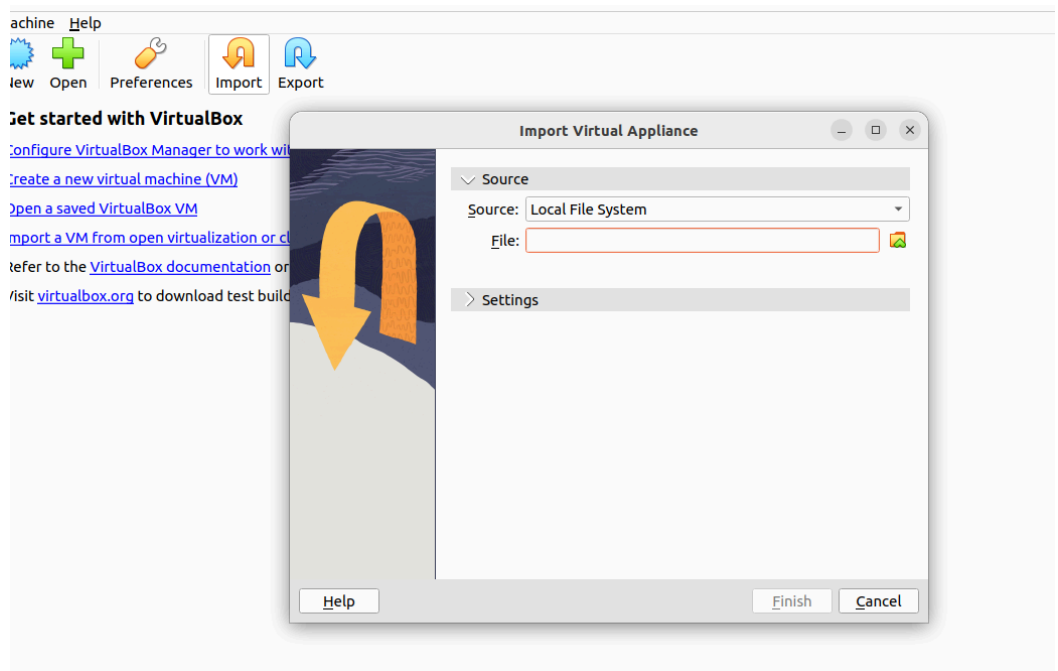


Figure 3-11: VirtualBox Import Appliance dialog

Step 2: Select the OVA

- Select the OVA downloaded in Section 3.4.
- Confirm that it matches the host architecture.

- Click Next.

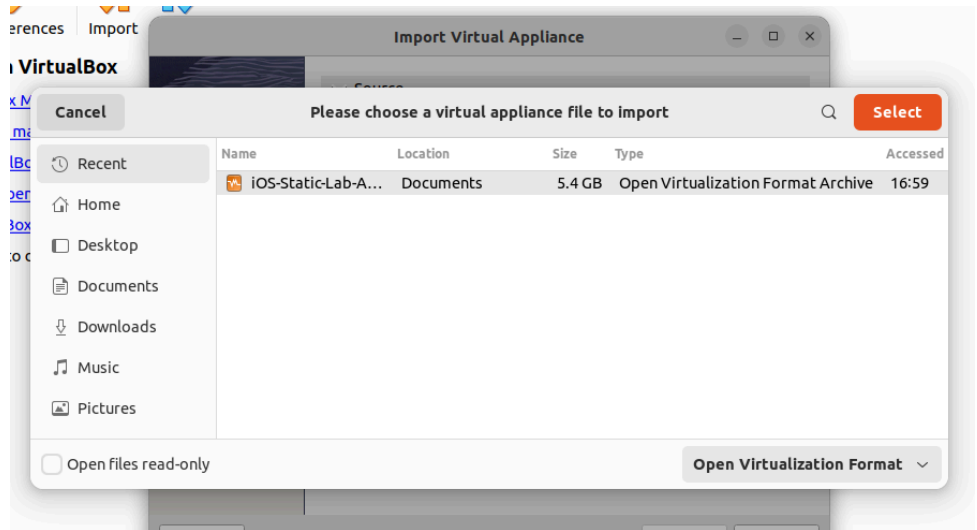


Figure 3-12: Selecting the laboratory OVA file

Step 3: Import the Appliance

- Review the displayed appliance information.
- Click **Import**.
- Do not change the displayed configuration unless instructed.

Step 4: Verify the Import

- Confirm that the imported **iOS-Static-Lab** virtual machine corresponding to the selected architecture appears in **VirtualBox Manager**.

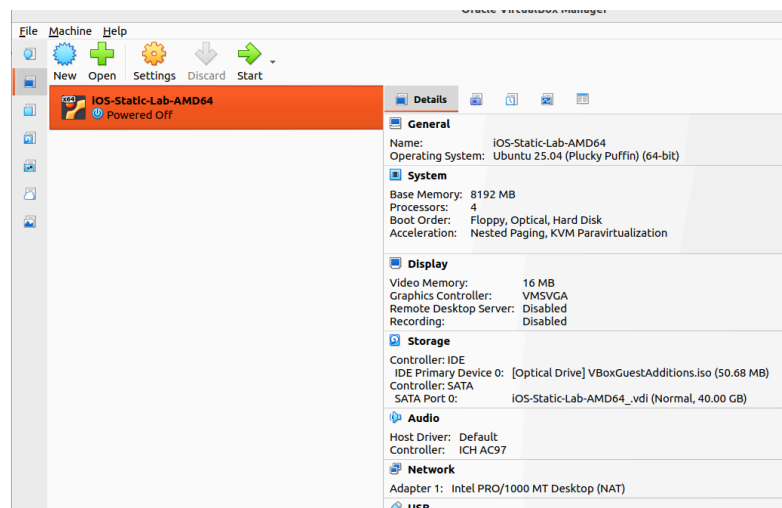


Figure 3-13-a: Imported laboratory virtual machine in VirtualBox Manager AMD Architecture

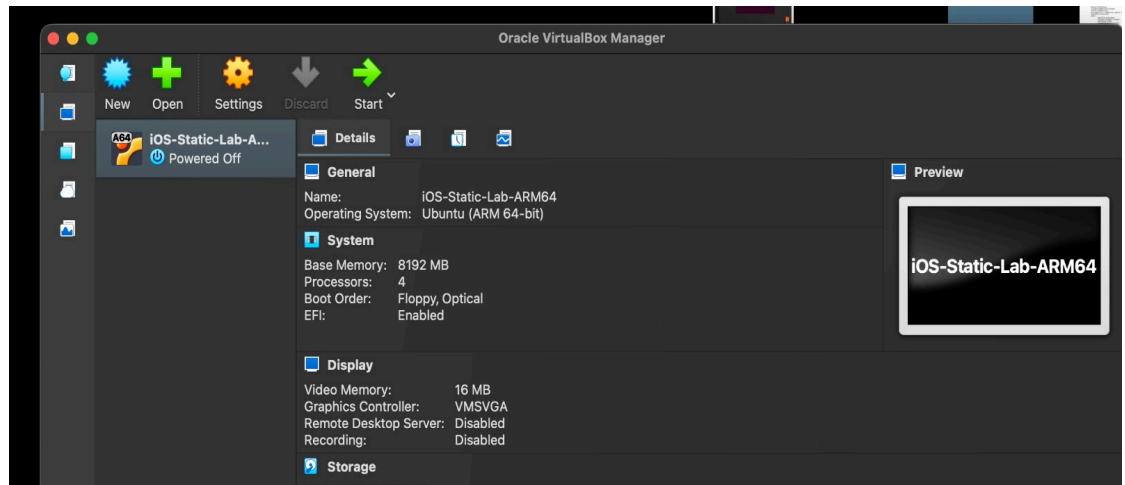


Figure 3-13-b: Imported laboratory virtual machine in VirtualBox Manager ARM Architecture

3.6 Start the Virtual Machine

Step 1: Select the Lab VM

- In **VirtualBox Manager**, select the imported laboratory virtual machine.
- Confirm that the VM corresponds to the selected architecture.

Step 2: Start the VM

- Click **Start**.

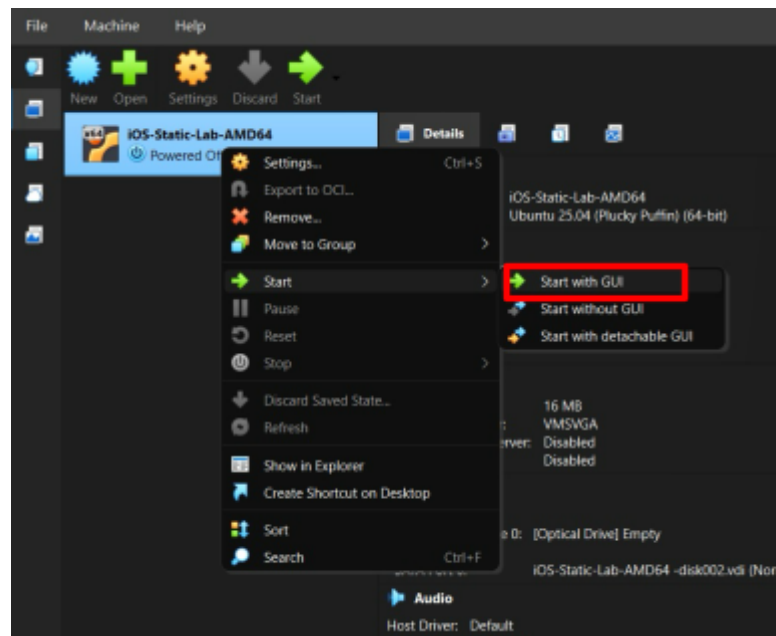


Figure 3-14: Starting the laboratory virtual machine

Step 3: Wait for Ubuntu

- Wait for the virtual machine to boot.
- The Ubuntu login screen should appear.

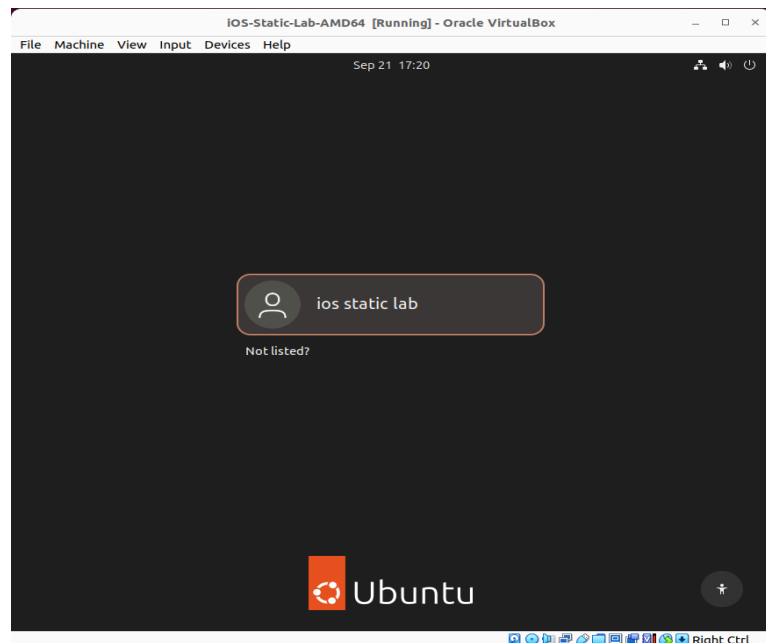


Figure 3-15: Ubuntu login screen

3.7 Log in to Ubuntu

Use the credentials provided for the laboratory environment.

Step 1: Log In

- Enter the provided **username** and **password**.
- Click **Log In**.

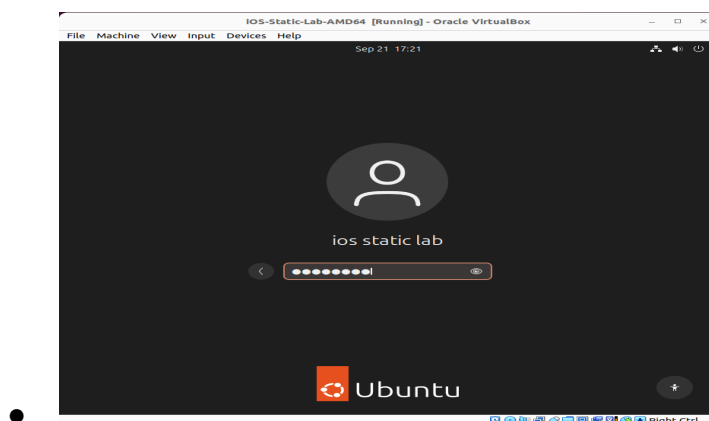


Figure 3-16: Ubuntu login screen

Step 2: Confirm the Desktop

- Wait for the Ubuntu desktop to load.

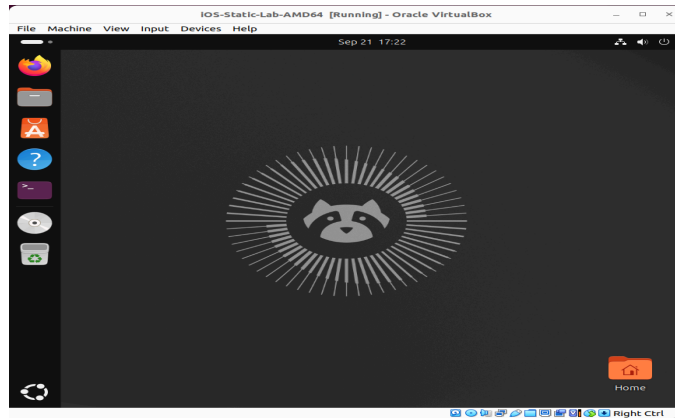


Figure 3-17: Ubuntu laboratory desktop

3.8 Verify the Lab Environment

Important: Run these commands **inside the Ubuntu virtual machine**, not on the Windows or Linux host.

Step 1: Verify Architecture

- Open **Terminal**.
- Run:
uname -m
- Confirm that the output is:

Selected OVA	Expected Output
AMD64 OVA	x86_64
ARM64 OVA	aarch64

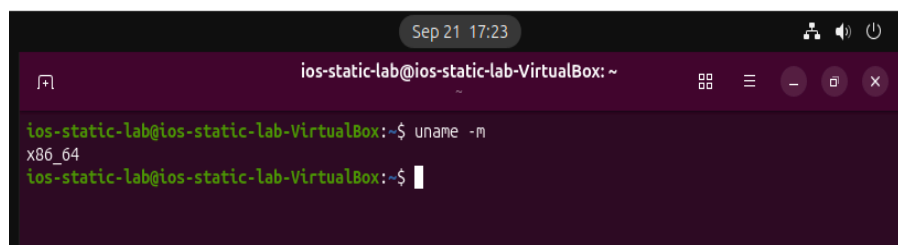


Figure 3-18: Lab VM architecture verification

4. Introduction to iOS Application Static Analysis

4.1 What is an IPA File?

- An IPA is an iOS application package.
- An IPA is an archive containing the files required for an iOS application. The file normally uses the .ipa extension.
- During this laboratory, the application may be provided as a ZIP archive containing the IPA package.
- For example:
 - **DVIA-v2.ipa.zip**
- After extraction, the important application content can be found under:
 - Payload/

4.2 IPA File Structure

A simplified IPA structure is:

```
Application.ipa
├── Payload/
│   ├── Application.app/
│   │   ├── Application
│   │   ├── Info.plist
│   │   ├── Frameworks/
│   │   ├── Resources
│   │   └── Other application files
```

4.3 iOS Application Bundle Structure

The .app directory is the application bundle.

Example:

```
Payload/
├── DVIA-v2.app/
│   ├── DVIA-v2
│   ├── Info.plist
│   ├── Frameworks/
│   ├── Resources
│   └── Other files
```

The application bundle contains the executable and supporting application resources.

4.4 Important Files and Directories

File / Directory	Purpose	Security Review
Main executable	Compiled application code	Binary analysis, APIs, strings
Info.plist	Application metadata and configuration	Bundle ID, ATS, URL schemes, permissions
Frameworks/	Embedded frameworks and libraries	Third-party components
.plist	Configuration/property data	Sensitive configuration
.json	Structured data/configuration	Possible sensitive information
.xml	Structured data/configuration	Application information
.db	Database	Possible stored data
.sqlite	SQLite database	Possible application data
.pem	Certificate/key-related file	Potentially sensitive material
.cer / .crt	Certificate-related files	Certificate configuration
.key	Key-related file	Potentially sensitive
Resources	Application resources	Information disclosure

Embedded frameworks	Libraries included with the application	Third-party and application functionality
---------------------	---	---

5. Static Analysis of an iOS Application

The guided analysis in this section uses the following application:

DVIA-v2.ipa.zip

The analysis starts with the IPA archive and progressively moves toward the application bundle and main executable.

The analysis should be performed in the following order:

IPA → Payload → Application Bundle → Info.plist → Files → Mach-O Binary → Headers → Symbols → Strings → URLs → Frameworks → APIs → Binary Protections → Security Review

Note: The screenshots provided in this manual are for **reference and demonstration purposes only**. The exact output, file names, directory paths, system information, and displayed values may differ depending on the user's system and laboratory environment. The commands shown in the manual perform the same function even when the displayed path or output is different. Users should follow the **command and procedure** rather than expecting the screenshot output to match exactly.

5.1 Identify the IPA File

The first step is to locate and verify the provided IPA archive. This confirms that the application package is available before beginning the analysis.

Command(s):

Navigate to the IPA Directory

```
cd ~/iOS-Static-Lab/IPA
```

Verify the Current Directory

```
pwd
```

List the Available Files

```
ls
```

Identify the File Type

```
file DVIA-v2.ipa.zip
```

```
ios-static-lab@ios-static-lab-VirtualBox:~$ cd ~/iOS-Static-Lab/IPA
pwd
ls -lh
file DVIA-v2.ipa.zip
/home/ios-static-lab/iOS-Static-Lab/IPA
total 45M
-rwxrwx--- 1 ios-static-lab ios-static-lab 26M Sep 17 14:21 Corellium_Cafe.ipa.zip
-rwxrwx--- 1 ios-static-lab ios-static-lab 9.3M Sep 17 14:17 DVIA-v2.ipa.zip
-rwxrwx--- 1 ios-static-lab ios-static-lab 10M Sep 17 14:21 'iGoat-Swift_v1.0_2.ipa.zip'
DVIA-v2.ipa.zip: IOS/iPadOS IPA file containing DVIA-v2.app/UT\011 Zip archive data, at least v1.0 to extract, compression method=store
```

Figure 5-1: Identification of the DVIA-v2 IPA archive

5.2 Inspect the IPA Archive

An IPA package is an archive containing the files required for an iOS application. Inspecting its contents before extraction helps the analyst understand the package structure and locate the application bundle.

Command(s):

unzip -l DVIA-v2.ipa.zip | head -30

```
ios-static-lab@ios-static-lab-VirtualBox:~/iOS-Static-Lab/IPA$ unzip -l DVIA-v2.ipa.zip | head -30
Archive:  DVIA-v2.ipa.zip
Length   Date    Time    Name
-----
0        2024-03-29 12:57  Payload/
0        2024-03-29 12:57  Payload/DVIA-v2.app/
0        2024-03-29 12:57  Payload/DVIA-v2.app/_CodeSignature/
54968    2024-03-29 12:56  Payload/DVIA-v2.app/_CodeSignature/CodeResources
4258     2024-03-29 12:56  Payload/DVIA-v2.app/main-bg.png
96316    2024-03-29 12:56  Payload/DVIA-v2.app/LaunchImage-1100-Portrait-2436h@3x.png
0        2024-03-29 12:57  Payload/DVIA-v2.app/Phishing.storyboardc/
```

Figure 5-2: Contents of the DVIA-v2 IPA archive

5.3 Extract the IPA

Extracting the archive makes the application files accessible for further examination. The files will be extracted into a separate directory to keep the original archive unchanged.

Command(s):

Extract the archive into a separate directory.

unzip -q DVIA-v2.ipa.zip -d DVIA-v2_Extracted

Verify the extracted contents:

ls DVIA-v2_Extracted

```
ios-static-lab@ios-static-lab-VirtualBox:~/iOS-Static-Lab/IPA$ unzip -q DVIA-v2.ipa.zip -d DVIA-v2_Extracted
ls -lah DVIA-v2_Extracted
total 12K
drwxrwxr-x 3 ios-static-lab ios-static-lab 4.0K Sep 17 17:06 .
drwxrwxr-x 3 ios-static-lab ios-static-lab 4.0K Sep 17 17:06 ..
drwxr-xr-x 3 ios-static-lab ios-static-lab 4.0K Mar 29 2024 Payload
```

Figure 5-3: Extracted IPA contents

5.4 Locate the Payload Directory

The **Payload** directory contains the iOS application bundle. Locating this directory is necessary to access the application's executable, configuration, and supporting files.

Command(s):

Move into the extracted directory:

cd DVIA-v2_Extracted

List its contents:

ls

Move into the Payload directory:

cd Payload

Verify the current directory:

pwd

List the contents:

ls

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA$ unzip -q DVIA-v2.ipa.zip -d DVIA-v2_Extracted
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA$ cd DVIA-v2_Extracted/Payload/DVIA-v2.app
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/DVIA-v2_Extracted/Payload/DVIA-v2.app$ ls -lah
total 9.0M
drwxrwx--- 1 root vboxsf 4.0K Mar 29 2024 .
drwxrwx--- 1 root vboxsf 4.0K Mar 29 2024 ..
drwxrwx--- 1 root vboxsf 4.0K Mar 29 2024 AntiAntiHookingDebugging.storyboardc
-rwxrwx--- 1 root vboxsf 7.0K Mar 29 2024 AppIcon60x60@2x.png
-rwxrwx--- 1 root vboxsf 8.7K Mar 29 2024 AppIcon76x76@2x~ipad.png
drwxrwx--- 1 root vboxsf 4.0K Mar 29 2024 AppIcons
drwxrwx--- 1 root vboxsf 4.0K Mar 29 2024 ApplicationPatching.storyboardc
-rwxrwx--- 1 root vboxsf 41K Mar 29 2024 Assets.car
```

Figure 5-4-b: Payload directory contents

5.5 Locate the Application Bundle

The **.app** directory is the iOS application bundle. It contains the main executable and the supporting files required by the application. Identifying the bundle establishes the working location for the remaining analysis.

Command(s):

For the guided DVIA-v2 analysis, move into the application bundle:

cd DVIA-v2.app

Verify the current directory:

pwd

List the bundle contents:

ls

```
ios-static-lab@ios-static-lab-VirtualBox:~/iOS-Static-Lab/IPA/DVIA-v2_Extracted/Payload/DVIA-v2.app$ ls -lah
total 9.0M
drwxr-xr-x 29 ios-static-lab ios-static-lab 4.0K Mar 29 2024 .
drwxr-xr-x 3 ios-static-lab ios-static-lab 4.0K Mar 29 2024 ..
drwxr-xr-x 2 ios-static-lab ios-static-lab 4.0K Mar 29 2024 AntiAntiHookingDebugging.storyboardc
-rw-r--r-- 1 ios-static-lab ios-static-lab 7.0K Mar 29 2024 AppIcon60x60@2x.png
-rw-r--r-- 1 ios-static-lab ios-static-lab 8.7K Mar 29 2024 AppIcon76x76@2x~ipad.png
drwxr-xr-x 3 ios-static-lab ios-static-lab 4.0K Mar 29 2024 AppIcons
drwxr-xr-x 2 ios-static-lab ios-static-lab 4.0K Mar 29 2024 ApplicationPatching.storyboardc
-rw-r--r-- 1 ios-static-lab ios-static-lab 41K Mar 29 2024 Assets.car
drwxr-xr-x 2 ios-static-lab ios-static-lab 4.0K Mar 29 2024 AttackingThirdPartyLibraries.storyboardc
drwxr-xr-x 3 ios-static-lab ios-static-lab 4.0K Mar 29 2024 Base.lproj
drwxr-xr-x 2 ios-static-lab ios-static-lab 4.0K Mar 29 2024 BinaryProtection.storyboardc
drwxr-xr-x 2 ios-static-lab ios-static-lab 4.0K Mar 29 2024 BrokenCryptography.storyboardc
drwxr-xr-x 2 ios-static-lab ios-static-lab 4.0K Mar 29 2024 ClientSideInjection.storyboardc
-rw-r-xr-x 1 ios-static-lab ios-static-lab 8.0M Mar 29 2024 DVIA-v2
```

Figure 5-4: DVIA-v2 application bundle contents

5.6 Analyze the Application Bundle

The application bundle should be reviewed before analyzing individual files. This provides an overview of the application's contents and helps identify files that may require further security review.

Command(s):

- Display the Bundle Structure
find . -maxdepth 2 -print | sort
- If the tree utility is available:
tree -L 2

```
ios-static-lab@ios-static-lab-VirtualBox:~/media/sf_iOS-Static-Lab/IPA/DVIA-v2_Extracted/Payload/DVIA-v2.app$ tree -L 2
.
├── AntiAntiHookingDebugging.storyboardc
│   ├── agV-Hr-I02-view-0oc-iy-UkR.nib
│   ├── Info.plist
│   └── UINavigationController-Jzx-8v-yuc.nib
├── AppIcon60x60@2x.png
├── AppIcon76x76@2x~ipad.png
├── AppIcons
│   ├── AppIcon.appiconset
│   ├── iTunesArtwork@1x.png
│   ├── iTunesArtwork@2x.png
│   └── iTunesArtwork@3x.png
├── ApplicationPatching.storyboardc
│   ├── 0pZ-Hv-cWv-view-lQD-88-22v.nib
│   ├── GNJ-HI-TXZ-view-aOG-mJ-icw.nib
│   ├── Info.plist
│   ├── UINavigationController-cbx-cQ-e6A.nib
│   └── UIViewController-0pZ-Hv-cWv.nib
└── Assets.car
```

Figure 5-5: DVIA-v2 application bundle structure

5.7 Analyze Info.plist

Info.plist is an application configuration file that contains metadata and settings used by iOS. Examining this file helps the analyst identify application details, URL schemes, App Transport

Security (ATS) settings, permission descriptions, and other configuration information relevant to the security review.

5.7.1 Display Info.plist

Command(s):

plistutil -p Info.plist

```
ios-static-lab@ios-static-lab-VirtualBox:~/media/sf_ios-Static-Lab/IPA/DVIA-v2_Extracted/Payload/DVIA-v2.app$ plistutil -p Info.plist
{
  "BuildMachineOSBuild": "23D60",
  "CFBundleDevelopmentRegion": "en",
  "CFBundleExecutable": "DVIA-v2",
  "CFBundleIcons": {
    "CFBundlePrimaryIcon": {
      "CFBundleIconFiles": [
        "AppIcon60x60"
      ],
      "CFBundleIconName": "AppIcon"
    }
  },
  "CFBundleIconName": "AppIcon"
}
```

Figure 5-6: Contents of Info.plist

5.7.2 Analyze Application Metadata

Command(s):

Search for common application metadata, including the **Bundle ID**, application name, and version information:

plistutil -p Info.plist | grep -E

"CFBundleIdentifier|CFBundleDisplayName|CFBundleName|CFBundleShortVersionString|CFBundleVersion"

```
ios-static-lab@ios-static-lab-VirtualBox:~/iOS-Static-Lab/IPA/DVIA-v2_Extracted/Payload/DVIA-v2.app$ plistutil -p Info.plist | grep -E "CFBundleIdentifier|CFBundleDisplayName|CFBundleName|CFBundleShortVersionString|CFBundleVersion"
  "CFBundleIdentifier": "com.hightitudehacks.DVIAswiftv2",
  "CFBundleName": "DVIA-v2",
  "CFBundleShortVersionString": "2.0",
  "CFBundleVersion": "1",
```

Figure 5-7: Application Metadata

5.7.3 Analyze App Transport Security

Command(s):

Search for the ATS configuration:

plistutil -p Info.plist | grep -A5 NSAppTransportSecurity

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_iOS-Static-Lab/IPA/Payload/DVIA-v2.app$ plistutil -p Info.plist | grep -A5 NSAppTransportSecurity
NSAppTransportSecurity: {
  "NSAppTransportSecurity": {
    "NSAllowsArbitraryLoads": true
  },
  "NSCameraUsageDescription": "To demonstrate the misuse of Camera, please grant it permission once.",
  "NSFaceIDUsageDescription": "The app needs FaceID permission",
  "UIAppFonts": [
```

Figure 5-8: App Transport Security Configuration

5.7.4 Analyze URL Schemes

Command(s):

Search for configured URL schemes:

plistutil -p Info.plist | grep -A10 CFBundleURLTypes

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_iOS-Static-Lab/IPA/Payload/DVIA-v2.app$ plistutil -p Info.plist | grep -A10 CFBundleURLTypes
CFBundleURLTypes: [
  {
    "CFBundleURLName": "com.hightitudehacks.DVIAswiftv2",
    "CFBundleURLSchemes": [
      "dvia",
      "dviaswift"
    ]
  }
],
"CFBundleVersion": "1",
"DTCompiler": "com.apple.compilers.llvm.clang.1_0",
```

Figure 5-9: Configured URL Schemes

5.7.5 Analyze Permission-related Entries

Command(s):

Search for common permission-related entries:

plistutil -p Info.plist | grep -Ei "UsageDescription|Camera|Location|Contacts|Photo|Microphone"

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_iOS-Static-Lab/IPA/Payload/DVIA-v2.app$ plistutil -p Info.plist | grep -Ei "UsageDescription|Camera|Location|Contacts|Photo|Microphone"
"NSCameraUsageDescription": "To demonstrate the misuse of Camera, please grant it permission once.",
"NSFaceIDUsageDescription": "The app needs FaceID permission",
```

Figure 5-10: Permission-Related Entries

5.7.6 Analyze iOS Version Information

Command(s):

plistutil -p Info.plist | grep -E "MinimumOSVersion"


```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ plistutil -p Info.plist | grep -E "MinimumOSVersion|DTPlatformVersion"
"DTPlatformVersion": "17.0",
"MinimumOSVersion": "12.0",
"UILaunchImageMinimumOSVersion": "11.0",
"UILaunchImageMinimumOSVersion": "8.0",
"UILaunchImageMinimumOSVersion": "8.0",
"UILaunchImageMinimumOSVersion": "7.0",
"UILaunchImageMinimumOSVersion": "7.0",
```

Figure 5-11: iOS Version Information

5.8 Search for Sensitive Files

Application bundles may contain configuration files, databases, certificates, keys, or other resources that require additional security review. Searching for these file types helps identify potential locations of sensitive information.

Command(s):

```
find . -type f \( -iname "*.plist" -o -iname "*.json" -o -iname "*.xml" -o -iname "*.db" -o -iname "*.sqlite" -o -iname "*.pem" -o -iname "*.cer" -o -iname "*.crt" -o -iname "*.key" -o -iname "*.p12" \)
```

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ find . -type f \( -iname "*.plist" -o -iname "*.json" -o -iname "*.xml" -o -iname "*.db" -o -iname "*.sqlite" -o -iname "*.pem" -o -iname "*.cer" -o -iname "*.crt" -o -iname "*.key" -o -iname "*.p12" \)
./ExcessivePermissions.storyboardc/Info.plist
./ApplicationPatching.storyboardc/Info.plist
./ClientSideInjection.storyboardc/Info.plist
./InsecureDataStorage.storyboardc/Info.plist
./JailbreakDetection.storyboardc/Info.plist
```

Figure 5-12: Sensitive-file search results

5.9 Identify the Main Mach-O Binary

The main application executable is a Mach-O binary, the native executable format used by iOS. Identifying it allows the analyst to perform binary-level analysis, including examining headers, symbols, strings, and security-related indicators.

Command(s):

- For DVIA-v2, verify the binary:

file DVIA-v2

- Check its size:

ls -lh DVIA-v2

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ file DVIA-v2
DVIA-v2: Mach-O 64-bit arm64 executable, flags=<NOUNDEFS|DYLDLINK|TWOLEVEL|WEAK_DEFINES|BINDS_TO_WEAK|PIE>
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ ls -lh DVIA-v2
-rwxrwx--- 1 root vboxsf 8.0M Mar 29 2024 DVIA-v2
```

Figure 5-13: Main Mach-O binary identification

5.10 Analyze Mach-O Headers

Mach-O headers and load commands contain structural information about the executable, including architecture, binary type, and other configuration details. Reviewing this information provides an initial understanding of the application binary and its protection-related characteristics.

Command:

llvm-objdump --macho --private-headers DVIA-v2 | head -50

```
ios-static-lab@ios-static-lab-VirtualBox: /media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-objdump --macho --private-headers DVIA-v2
DVIA-v2:
Mach header
  magic cputype cpusubtype  caps   filetype ncmds sizeofcmds   flags
MH_MAGIC_64  ARM64      ALL    0x00    EXECUTE   74     8480   NOUNDEFS DYLDLINK TWOLEVEL WEAK_DEFINES BINDS_TO_WEAK PIE
Load command 0
  cmd LC_SEGMENT_64
  cmdsize 72
  segname __PAGEZERO
  vmaddr 0x0000000000000000
  vmsize 0x0000000010000000
  fileoff 0
```

Figure 5-14: Mach-O header information

5.11 Analyze Imported Symbols

Imported symbols indicate functions that the application binary references from external libraries or system frameworks. Reviewing these symbols helps identify functionality related to cryptography, storage, networking, WebViews, and other security-relevant areas.

Command(s):

llvm-nm -u DVIA-v2

For an initial review:

llvm-nm -u DVIA-v2 | head -40

```
ios-static-lab@ios-static-lab-VirtualBox: /media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-nm -u DVIA-v2
_$$10Foundation10URLRequestV10httpMethodSSSgvs
_$$10Foundation10URLRequestV19_bridgeToObjectiveCSO12NSURLRequestCyF
App Center_ion10URLRequestV3url11cachePolicy15timeoutIntervalAcA3URLV_So017NSURLRequestCacheE0VSDtcfc
_$$10Foundation10URLRequestV8httpBodyAA4DataVSgvs
_$$10Foundation10URLRequestV8setValue_18forHTTPHeaderFielddySSSg_SSTF
_$$10Foundation10URLRequestVMa
_$$10Foundation12URLQueryItemV4name5valueACSSh_SSSghtcfc
_$$10Foundation12URLQueryItemVMa
_$$10Foundation12URLQueryItemVs23CustomStringConvertibleAAMc
_$$10Foundation14DateComponentsV8calendar8timeZone3era4year5month3day4hour6minute6second10nanosecond7weekday007Ordinal7quarter1
1weekOfMonths4Year0h7ForWeeksU0AcA8CalendarVSg_AA04TimeF0VSgSiSgA13Ztcfc
```

Figure 5-15: Imported symbols

5.12 Extract Strings

Strings are readable text embedded in the application binary. They may reveal URLs, domain names, error messages, configuration values, debug messages, and other application-specific information that can support further analysis.

Command(s)

strings DVIA-v2

For an initial review:

strings DVIA-v2 | head -50

```
ios-static-lab@ios-static-lab-VirtualBox: /media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ strings DVIA-v2 | head -50
__PAGEZERO
__TEXT
__text
__TEXT
__stubs
__TEXT
__stub_helper
__TEXT
__objc_stubs
__TEXT
__cstring
TEXT
```

Figure 5-16: Strings extracted from the application binary

5.13 Search for Potential Secrets

Searching extracted strings for terms associated with credentials and secrets helps identify values that may require further investigation. A matching string is only an indicator; its context and purpose must be reviewed before determining whether it represents an actual vulnerability.

Command(s):

strings DVIA-v2 | grep -iE 'password|passwd|secret|api[_-]?key|token|private[_-]?key|bearer'

```
ios-static-lab@ios-static-lab-VirtualBox: /media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ strings DVIA-v2 | grep -iE 'password
|passwd|secret|api[_-]?key|token|private[_-]?key|bearer'
aPassword := nil
Both password and the key (%d) or HMACKey (%d) are set.
%s -- Cannot be used in OpenSSL mode. An IV or password is required
dvpassword
-[RNOpenSSLDecryptor initWithPassword:handler:]
+[RNOpenSSLDecryptor decryptData:withPassword:error:]
aPassword
Unable to create AES key for password: %d
realmPasswordTextField
T@"UITextField",N,&VrealmPasswordTextField
passwordTextField
T@"UITextField",N,&VpasswordTextField
```

Figure 5-17: Potential secret-related strings

5.14 Extract URLs

URLs embedded in the application binary may reveal API endpoints, external services, development servers, and other network destinations. Identifying these URLs provides useful leads for reviewing the application's network configuration.

Command(s):

strings DVIA-v2 | grep -Eo 'https?:/[^\s:;<>"]+'

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_iOS-Static-Lab/IPA/Payload/DVIA-v2.app$ strings DVIA-v2 | grep -Eo 'https?:/[^\[:space:]]*<["'']>+'
https://github.com/yapstudios/YapDatabase/wiki/Views
https://github.com/yapstudios/YapDatabase/wiki/Views
http://example.com/
https://example.com/
https://example.com/
http://highaltitudehacks.com
http://highaltitudehacks.com/2013/11/08/ios-application-security-part-21-arm-and-gdb-basics
https://www.youtube.com/watch?v=HsV6jaA5J2I
http://www.thejuniperfund.org/
http://damnvulnerableiosapp.com
https://twitter.com/prateekg147
```

Figure 5-18: URLs identified in the application binary

5.15 Analyze Embedded Frameworks

The **Frameworks** directory may contain libraries and third-party frameworks used by the application. Identifying embedded frameworks helps the analyst understand which additional components are included in the application bundle and may require security review.

Command:

ls Frameworks

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_iOS-Static-Lab/IPA/Payload/DVIA-v2.app$ ls -lah Frameworks
total 24K
drwxrwx--- 1 root vboxsf 4.0K Mar 29  2024 .
drwxrwx--- 1 root vboxsf 4.0K Sep 16 17:39 ..
drwxrwx--- 1 root vboxsf 4.0K Sep 16 17:39 Bolts.framework
drwxrwx--- 1 root vboxsf 4.0K Sep 16 17:39 Parse.framework
drwxrwx--- 1 root vboxsf 4.0K Sep 16 17:39 Realm.framework
drwxrwx--- 1 root vboxsf 4.0K Sep 16 17:39 RealmSwift.framework
```

Figure 5-19: Embedded frameworks

5.16 Analyze Cryptographic APIs

Cryptographic API references provide an initial indication of the cryptographic functionality used by the application. Reviewing these references helps identify areas for further examination of encryption, hashing, key handling, and random-number generation.

Command:

llvm-nm -u DVIA-v2 | grep -iE 'CCCrypt|SecKey|SecRandom|MD5|SHA1|SHA256|SHA512'

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_iOS-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-nm -u DVIA-v2 | grep -iE 'CCCrypt
pt|SecKey|SecRandom|MD5|SHA1|SHA256|SHA512'
_CCCrypt
_CCCryptorCreate
_CCCryptorFinal
_CCCryptorGetOutputLength
_CCCryptorRelease
_CCCryptorUpdate
```

Figure 5-20: Cryptographic API imports

5.17 Analyze Storage APIs

Storage-related API references help identify how the application may interact with local storage, including Keychain services, user defaults, SQLite databases, and file-system operations. These indicators can guide further review of how application data is stored and protected.

Command(s):

```
llvm-nm -u DVIA-v2 | grep -iE 'SecItem|NSUserDefaults|sqlite3_|FileManager'
```

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-nm -u DVIA-v2 | grep -iE 'SecItem|NSUserDefaults|sqlite3_|FileManager'
_OBJC_CLASS_$_NSUserDefaults
_OBJC_CLASS_$_FileManager
_SecItemAdd
_SecItemCopyMatching
_SecItemDelete
_SecItemUpdate
```

Figure 5-21: Storage-related API imports

5.18 Analyze WebView APIs

WebView-related API references indicate that the application may display or interact with web content. Identifying these references helps locate functionality that may require further review, including JavaScript execution and web content loading.

Command(s):

```
llvm-nm -u DVIA-v2 | grep -iE 'WKWebView|UIWebView|evaluateJavaScript|loadRequest'
```

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-nm -u DVIA-v2 | grep -iE 'WKWebView|UIWebView|evaluateJavaScript|loadRequest'
_OBJC_CLASS_$_WKWebView
_OBJC_CLASS_$_WKWebViewConfiguration
```

Figure 5-22: WebView-related API imports

5.19 Analyze Network and TLS APIs

Network and TLS-related API references help identify functionality associated with network communication, certificate handling, trust evaluation, and authentication challenges. These findings can guide further examination of the application's network security implementation.

Command(s):

```
llvm-nm -u DVIA-v2 | grep -iE 'NSURLSession|NSURLSessionConfiguration|SecTrust|SecCertificate|AuthenticationChallenge'
```

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-nm -u DVIA-v2 | grep -iE 'NSURLSession|NSURLSessionConfiguration|SecTrust|SecCertificate|AuthenticationChallenge'
_OBJC_CLASS_$_NSURLSession
_OBJC_CLASS_$_NSURLSessionConfiguration
_SecCertificateCopyData
```

Figure 5-23: Network and TLS-related API imports

5.20 Analyze PIE Protection

Position Independent Executable (PIE) is a binary protection mechanism that allows an executable to be loaded at varying memory addresses. Checking PIE-related information provides an initial indication of whether this protection is reflected in the application binary.

Command(s):

Check the Mach-O information for PIE-related information:

```
llvm-objdump --macho --private-headers DVIA-v2 | grep -i PIE
```

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-objdump --macho --private-headers DVIA-v2 | grep -i PIE
MH_MAGIC_64 ARM64 ALL 0x00 EXECUTE 74 8480 NOUNDEFS DYLDLINK TWOLEVEL WEAK_DEFINES BINDS_TO_WEAK PIE
```

Figure 5-24: PIE analysis

5.21 Analyze Encryption Information

Mach-O encryption-related load commands may contain information about the encryption status of an executable. Examining these fields provides an initial view of the encryption information recorded in the binary.

Command(s):

```
llvm-objdump --macho --private-headers DVIA-v2 | grep -A10 -E 'LC_ENCRYPTION_INFO|LC_ENCRYPTION_INFO_64'
```

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-objdump --macho --private-headers DVIA-v2 | grep -A10 -E 'LC_ENCRYPTION_INFO|LC_ENCRYPTION_INFO_64'
cmd LC_ENCRYPTION_INFO_64
  cmdsize 24
  cryptoff 32768
  cryptsize 4243456
  cryptid 0
```

Figure 5-25: Mach-O encryption information

5.22 Analyze Stack Protection

Stack protection mechanisms help detect certain types of stack-based memory corruption. Searching for stack-canary-related symbols provides an initial indication of whether the binary references common stack-protection functions.

Command(s):

```
llvm-nm -u DVIA-v2 | grep -E '__stack_chk_fail|__stack_chk_guard'
```

```
ios-static-lab@ios-static-lab-VirtualBox:/media/sf_ios-Static-Lab/IPA/Payload/DVIA-v2.app$ llvm-nm -u DVIA-v2 | grep -E '__stack_chk_fail|__stack_chk_guard'
__stack_chk_fail
__stack_chk_guard
```

Figure 5-26: Stack protection analysis

6. Identification of Potential Security Weaknesses

In Section 5, we examined different parts of an iOS application, including its configuration file, executable, strings, APIs, and other files.

During this analysis, we may discover information or application features that require further security review.

For example, an application may contain a password in its executable, use an insecure network configuration, or store sensitive information in an unsuitable location.

These observations may indicate potential security weaknesses.

Important: Finding something during static analysis does not automatically mean that the application is vulnerable. The analyst must understand what was found, why it matters, and whether it creates a security risk.

The following subsections explain common areas that should be reviewed during iOS application static analysis.

6.1 Sensitive Information Exposure

What does this mean?

Sensitive information exposure occurs when an application contains information that should not be unnecessarily accessible to others.

An iOS application may include information inside its executable, configuration files, or other resources. If sensitive information is included in the application package, someone who obtains the IPA may be able to extract and examine it.

What should we look for?

Examples of information that may require security review include:

- Internal server URLs or API endpoints.
- Usernames or passwords.
- Authentication tokens.
- API keys.
- Internal configuration details.
- Debug messages containing sensitive information.

Why is this a security concern?

If sensitive information is exposed in an application package, an attacker who obtains the IPA may be able to examine it without running the application.

Depending on the information exposed, this could reveal internal application details or provide access to protected resources.

Practical connection to Section 5

Use the results collected during:

- Section 5.8 – Search for Sensitive Files
- Section 5.12 – Extract Strings
- Section 5.13 – Search for Potential Secrets

- Section 5.14 – Extract URLs

Remember: Not every URL, string, or configuration value is sensitive. Review its purpose before identifying it as a security weakness.

6.2 Hardcoded Secrets

What does this mean?

A hardcoded secret is a sensitive value written directly into an application's source code or compiled executable.

For example, a developer might include an API key or password directly in the application instead of obtaining or managing it securely.

When the application is compiled, some of these values may remain inside the executable or other application files.

What should we look for?

Examples of potential hardcoded secrets include:

- API keys.
- Passwords.
- Authentication tokens.
- Private keys.
- Service credentials.

Why is this a security concern?

An IPA can be extracted and examined using static-analysis tools.

If a valid secret is embedded in the application, someone who obtains the IPA may be able to recover it.

Depending on the permissions associated with that secret, it could potentially be misused to access protected services or information.

Practical connection to Section 5

Refer to:

- Section 5.12 – Extract Strings
- Section 5.13 – Search for Potential Secrets

When a suspicious value is identified, the analyst should check:

1. What is the value?
2. Where was it found?
3. Does it appear to be a secret or just an example?
4. Is it intended to be public?

5. Could it provide access to a protected resource?

Remember: A string containing the word **password** is not necessarily a hardcoded password. Verify the actual value and its purpose before reporting a finding.

6.3 Insecure Network Configuration

What does this mean?

An iOS application communicates with servers to send or receive information.

For example, an application may connect to a server to log in a user, retrieve data, or submit a request.

If the application is configured to communicate insecurely, sensitive information may be exposed or communication may be vulnerable to interception or manipulation.

What should we look for?

The analyst should review:

- HTTP URLs that may indicate unencrypted communication.
- App Transport Security (ATS) settings.
- Network-related configuration.
- Certificate-handling references.
- TLS and trust-evaluation references.
- Development or internal server endpoints.

Why is this a security concern?

If sensitive information is transmitted without appropriate protection, an attacker in a suitable network position may be able to observe or manipulate the communication.

For example, transmitting login credentials over unencrypted HTTP could expose them to others who can capture the traffic.

Practical connection to Section 5

Refer to:

- Section 5.7.3 – Analyze App Transport Security
- Section 5.14 – Extract URLs
- Section 5.19 – Analyze Network and TLS APIs

Remember: Finding an HTTP URL or an ATS exception is an indication for further review. The analyst must determine whether the application actually uses the endpoint and whether sensitive data is transmitted insecurely.

6.4 Weak Cryptographic Usage

What does this mean?

Cryptography is used to protect information through encryption, hashing, and other security operations.

An application may use cryptographic functions to protect passwords, sensitive files, communication, or other data.

However, using a cryptographic API does not automatically mean that the application is secure. The algorithm, implementation, and purpose of the operation must also be considered.

What should we look for?

The analyst should review references to:

- Weak or outdated cryptographic algorithms.
- Hashing functions such as MD5 or SHA-1.
- Encryption and decryption functions.
- Key-generation and key-management functions.
- Random-number generation functions.

Why is this a security concern?

If an application uses unsuitable cryptographic algorithms or handles encryption keys insecurely, its data protection may be weakened.

For example, using an unsuitable hashing algorithm for password storage may make it easier for an attacker to recover passwords if the stored hashes are obtained.

Practical connection to Section 5

Refer to:

- Section 5.11 – Analyze Imported Symbols
- Section 5.16 – Analyze Cryptographic APIs

Remember: Finding an MD5, SHA-1, or encryption API reference does not automatically prove a vulnerability. The analyst must determine how the function is used and whether its use is appropriate for the security requirement.

6.5 Insecure Storage Indicators

What does this mean?

iOS applications may store information locally on a device.

For example, an application may save user preferences, login-related information, downloaded files, or application data.

If sensitive information is stored without appropriate protection, someone who gains access to the stored data may be able to read or misuse it.

What should we look for?

The analyst should identify references to storage mechanisms such as:

- UserDefaults.
- Local files.
- SQLite databases.
- Keychain services.
- Other application storage mechanisms.

Examples of potentially sensitive stored information include:

- Authentication tokens.
- Personal information.
- Login-related data.
- Financial information.
- Application secrets.

Why is this a security concern?

The security risk depends on what information is stored and how it is protected.

For example, storing a sensitive authentication token in an unsuitable location may expose it if the stored data becomes accessible to an attacker.

Practical connection to Section 5

Refer to:

- Section 5.8 – Search for Sensitive Files
- Section 5.17 – Analyze Storage APIs

Remember: Using UserDefaults or SQLite is not automatically a vulnerability. The analyst must determine what data is stored, where it is stored, and whether the protection is appropriate for that data.

6.6 WebView Security Indicators

What does this mean?

A WebView allows an iOS application to display web content inside the application itself.

For example, an application may use a WebView to display a webpage, load online content, or provide web-based functionality without opening a separate browser.

If WebView functionality is not implemented securely, it may expose the application to risks associated with untrusted web content or unsafe interactions between web content and native application code.

What should we look for?

The analyst should review references to:

- WebView components.

- JavaScript execution.
- Web content loading.
- Navigation handling.
- Communication between web content and native application code.

Why is this a security concern?

If an application loads untrusted content or exposes sensitive native functionality to web content without appropriate restrictions, an attacker may be able to misuse that functionality.

For example, an insecure JavaScript-to-native interface could potentially expose sensitive application functionality to untrusted web content.

Practical connection to Section 5

Refer to:

- Section 5.11 – Analyze Imported Symbols
- Section 5.18 – Analyze WebView APIs

Remember: Finding a WebView or JavaScript-related API does not automatically indicate a vulnerability. The analyst must examine how the WebView is configured, what content it loads, and how it interacts with native application functionality.

6.7 Sensitive Files

What does this mean?

An iOS application bundle may contain different types of files that support the application's functionality.

Some files may contain configuration details, stored information, certificates, or other data that requires security review.

Searching for these files helps the analyst identify locations where sensitive information might be present.

What should we look for?

File Extension	What it may contain
.plist	Application configuration and metadata
.json	Structured data or configuration
.xml	Structured data or application settings

.db	Database records
.sqlite	SQLite database records
.pem	Certificates or cryptographic key material
.cer / .crt	Certificates
.key	Potential cryptographic key material
.p12	Certificate and potentially private-key material

Why is this a security concern?

If an application package contains sensitive information that should not be publicly accessible, someone who obtains the IPA may be able to extract and examine those files.

For example, a database included in the application package might contain sensitive information, or a private key might be exposed if it is improperly bundled.

Practical connection to Section 5

Refer to:

- Section 5.6 – Analyze the Application Bundle
- Section 5.8 – Search for Sensitive Files

When a file is identified, the analyst should:

1. Identify the file and its location.
2. Determine what type of information it contains.
3. Check whether the information is sensitive.
4. Determine whether the information should be included in the application package.
5. Document any security concerns with supporting evidence.

Remember: A file extension alone does not establish a vulnerability. For example, a **.cer** certificate is not necessarily secret, and a **.key** file name does not prove that the file contains a sensitive private key.